Jobs: A Real Promise or Really Fake?



Ahsan Tebha

Dr. Young

MIS 769

10-31-26

Contents

Abstract 3
Research Question 3
Methodology & Analysis 4
    Methodology 4
    Analysis 6
    Limitations 6
Conclusion 8
References 8

Abstract

This study examines whether the language of a job posting can predict whether the posting is fraudulent, and which words drive that prediction. Using a Kaggle dataset of 17,880 job postings, of which 866 (4.84%) were labeled fraudulent, I combined the title, company profile, description, requirements, and benefits into a single text field for each posting. The text was cleaned by removing URLs, numbers, punctuation, and stop words, then converted into 5,000 TF-IDF features using single words and two-word phrases. I split the data 80/20 while keeping the fraud rate the same in both sets, and trained a logistic regression model with class weights to account for the imbalance between real and fraudulent postings.

The language of a posting was a strong predictor of fraud. The model caught 90% of the fraudulent postings in the test set, with a precision of 0.64, an F1 score of 0.75, and a PR-AUC of 0.89 compared to 0.05 for random guessing. Accuracy was 97%, but this was only slightly above the 95% that would come from labeling every posting as real, so I relied on the fraud-class metrics. Fraudulent postings were also shorter, with a median of about 157 cleaned words versus 233 for real postings. The words pointing to fraud centered on easy money (earn, money, cash), entry-level administrative work (data entry, clerk), and contact outside the platform (link, email, phone). Real postings leaned on company and culture language such as team, clients, growing, and startup, along with technology and marketing terms.

These findings suggest that fraudulent postings can be separated from real ones largely by how they describe the work and the employer. However, the model only reaches about 64% precision, so it is better suited to flagging postings for review than to automatically removing them. Some of the strongest words, such as the industry terms, may come from repeated scam templates and may not carry over to other sources. Future work should compare other models, check how many postings contain each top word, and combine the text with posting details like company logo and salary. Like the sample, these results show association in this dataset, not cause.

Research Question

The research question I examined is, “Can the language of a job posting predict whether it is fraudulent, and which words drive that prediction?”

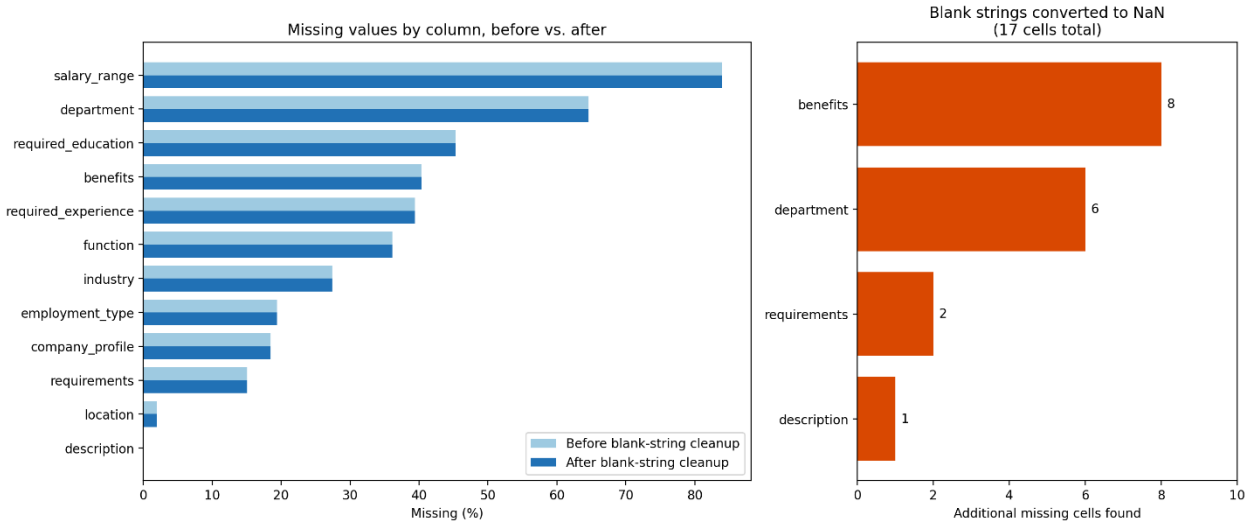
This research question has two parts: a predictive part of how well a model can classify postings as real or fake using only text. The second part is which words or phrases are the most associated with real and fake posts?

Methodology & Analysis

Methodology

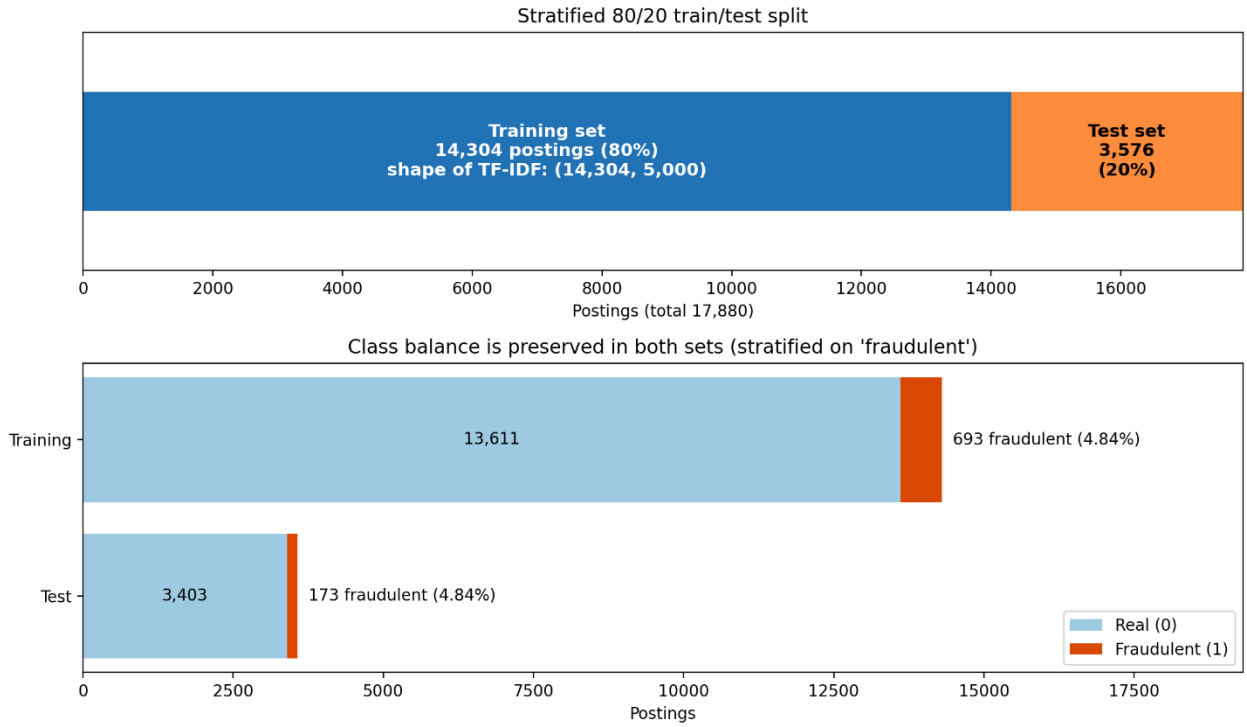
The method I used was to find a dataset on Kaggle[1] containing 17,880 job postings, each labeled as real or fraudulent. The dataset is one row per posting, with the title, company profile, description, requirements, benefits, and several details like employment type and required education. I first looked at the top rows, the shape, and the column names to confirm the data loaded correctly. I then checked the balance of the fraudulent label, which showed 17,014 real postings (95.16%) and only 866 fraudulent ones (4.84%). Because of this imbalance, accuracy alone would be misleading, since a model that called every posting real would already score about 95%. This check shaped the rest of the method. I used a stratified split, class weights in the model, and judged the results on precision, recall, F1, and PR-AUC (area under the precision-recall curve) for the fraudulent class.

Next, I checked for missing values. Salary range was missing in nearly 84% of postings, department in about 65%, and several other columns between 15% and 45%. Since empty strings can hide real missing data, I replaced blank and whitespace-only values with NaN and compared the counts before and after. This only uncovered 17 additional cells (8 in benefits, 6 in department, 2 in requirements, and 1 in description), so the original counts were mostly accurate. The check mattered because the question was about language. Description was the main text field, and it had only 2 missing rows, so the missing data was not a threat to the analysis. I treated missing text as empty text in the next step instead of filling it with a placeholder like "Unknown," because the model would have learned that placeholder as if it were a real word. Figure 1 below.



To answer the question, I combined the title, company profile, description, requirements, and benefits into one text field per posting. I confirmed this worked by looking at the first rows, where each posting started with its title followed by the rest of the text. I then cleaned the text by making it lowercase, removing URLs, HTML tags, numbers, and punctuation, and dropping stop words and words shorter than three letters. I compared the raw and cleaned text to make sure the cleaning was not removing anything important. It did remove some things, like “DC” from “Washington DC” and the “52” in “Food52,” which I accepted as a limitation. I also checked that no posting was left empty after cleaning, which was zero, so every posting could be used. While doing this I found that the median cleaned posting was 156.5 words for fraudulent postings versus 233 for real ones, an early sign that the two groups differ in how they are written.

I split the data 80/20 into 14,304 training and 3,576 test postings. The split was stratified on the fraud label so both sets would keep the same fraud rate. This mattered because fraud was rare, and a random split could have left too few fraudulent postings in the test set. The test set contained 173 fraudulent postings. I then converted the text into 5,000 TF-IDF features using single words and two-word phrases, learning the vocabulary from the training set only. This check was about leakage, since fitting on the test set would let information from it influence the model. The output shape of (14,304, 5,000) confirmed one row per training posting and the expected number of features. Figure 2 below.



Finally, I trained a logistic regression model with balanced class weights and evaluated it on the test set. Because accuracy was not reliable here, I looked at the fraudulent row of the

classification report, a confusion matrix, and PR-AUC, which I compared with the 0.048 a random model would score. To answer the second half of the question, I used the model's coefficients to find the 20 words pushing most strongly toward fraud and the 20 pushing toward real postings, since logistic regression weights can be read directly. I did not run a check on how many postings contain each top word, so the weights should be read as association in this dataset, not cause.

Analysis

The results show that the language of a job posting is a strong predictor of whether it is fraudulent. The model caught 90% of the fraudulent postings in the test set (recall of 0.90), with a precision of 0.64, an F1 score of 0.75, and a PR-AUC of 0.89, compared with the 0.05 a random model would score. Overall accuracy was 97%, but a model that labeled every posting as real would have scored 95.16%, so I didn't treat accuracy as the evidence. The fraud-class metrics and PR-AUC were the meaningful measures, and they show that the text carried real information beyond the class imbalance.

The words behind the predictions formed a recognizable pattern. Fraudulent postings pointed to easy money (earn, money, cash, work home), entry-level administrative work (data entry, clerk, administrative), and contact outside the platform (link, email, phone). The word "link" had the strongest weight by a wide margin. Real postings leaned on language about the company and its culture (team, clients, growing, startup, fun) along with technology and marketing terms (software, web, digital, marketing). Fraudulent postings were also shorter, with a median of about 157 cleaned words versus 233 for real postings. Together, these results suggest that legitimate employers describe who they are and what the work involves, while fraudulent postings stress how simple the job is and how quickly someone can be paid.

The precision of 0.64 changes how the model should be used. Since recall was 0.90, the model flagged roughly 244 postings to find about 156 of the 173 fraudulent ones, which means about 88 real postings were flagged by mistake. The model is better suited as a screening tool that sends postings to a person for review than as a filter that removes postings automatically.

Limitations

Number of fraudulent postings. The dataset had 866 fraudulent postings out of 17,880, and only 173 of them were in the test set. The fraud-class metrics depend on those 173 cases, so a few postings classified differently would move precision and recall by a point or more. I ran one model on one random split, so I cannot say how much the results would shift on a different split. I also cannot say how stable the top words are, since a weight can be driven by a small number of postings. A few hundred fraud cases are also small for the 5,000 features the model had to learn from, which raises the chance it learned patterns specific to this sample and means there can be some underrepresentation in the data.

Row count and repeated templates. 17,880 rows is a modest size, and fraud postings often come in batches of near-identical text from the same scammer. If many fraudulent postings share a template, the model can learn the template instead of the language of fraud, and it would score well on a test set that contains copies of the same templates. The industry words (oil gas, offshore, engineering, hospital) may be an example of this. I did not check how many postings contain each top word, so I cannot separate broad patterns from small clusters. I also did not find duplicates beyond the check I ran, and near-duplicates would not have been caught by an exact-match check.

How the labels were created. The fraudulent label is the dataset's own, and I did not have information on how postings were judged to be fake. If the labels came from user reports or moderator review, then fraud that was never caught or reported is labeled as real. That would make the real class noisy, and the model's measured precision look worse than it is, or better, depending on what was missed.

No tracking of reposting or what happened after. Each row is a single snapshot. The data does not show whether a posting was later reposted, how long it stayed up, or whether the same posting appeared under different companies. Repeated reposting of the same role is a signal a real fraud check would use, and this analysis has none of it. The reverse problem also applies. A posting can be legitimate and still look suspicious, such as when funding for the role was pulled, the position was frozen, or the company kept an old listing active to collect applications. Those postings could look like fraud in practice but are labeled as real here, and the model cannot tell the difference between a dead real posting and a fake one.

External factors the text cannot show. The model only sees words. It has no information about whether the company exists, whether the contact email matches the company's domain, or how the applicant was treated after applying. Scammers can also change their wording once the patterns are known, so a model trained on today's postings may weaken over time. The dataset does not state when the postings were collected or which platform they came from, so I cannot say whether the results apply to other sites or other years.

Choices I made in the method. Removing numbers and words shorter than three letters discarded signals like salary amounts and abbreviations such as "HR" or "IT." I used one model (logistic regression) with TF-IDF and did not compare it with other approaches. I also left the metadata columns (company logo, salary, telecommuting) out of the model on purpose, since the question was about language, so I cannot say how much predictive power the text adds on top of them.

Association, not cause. The word weights show which words appear more often in fraudulent postings in this dataset. They do not show that a word makes a posting fraudulent, and

a legitimate posting that uses words like “earn” or “work home” would still be pushed toward the fraud class.

Conclusion

The language of a job posting can predict fraud well enough to be useful for screening, and the words that drive the prediction describe easy pay, low barriers, and contact outside the platform. However, these results come from a small number of fraud cases, a single snapshot of the data, and labels I could not verify. They show a pattern in this dataset, not a rule that would hold for every job board or time period. A stronger version of this work would add repost and posting-history data, check the labels against how they were assigned, remove repeated templates before splitting, checking a posting across multiple job boards, and test across multiple random splits.

References

https://www.kaggle.com/datasets/shivamb/real-or-fake-fake-jobposting-prediction

Grammerly

Claude Sonnet 5.5